Indexes: B-tree vs LSM-tree

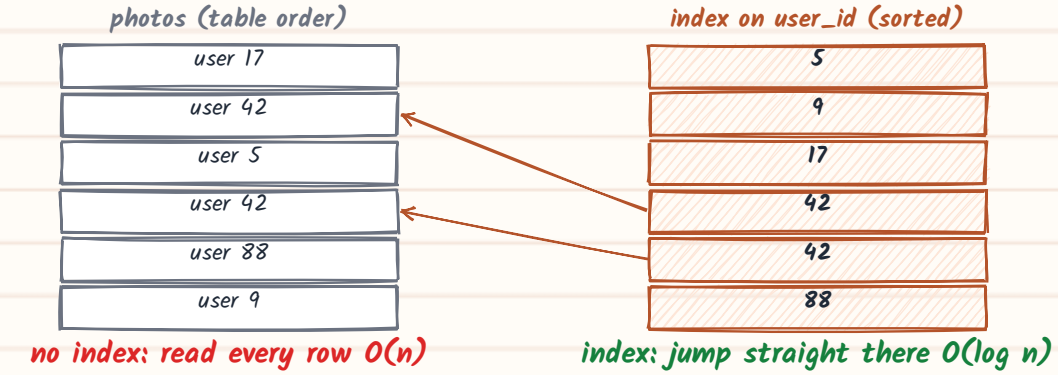
Lesson 11 · how databases find data fast

In this lesson → ① Slow query ② What an index is ③ B-tree ④ B-tree costs ⑤ LSM writes ⑥ LSM reads ⑦ Compare ⑧ Using indexes ⑨ Interview Q&A

Running project: SnapLink, our photo-sharing app that grows from 1 server to 100M users.

> An **index** is a separate, **sorted** structure on one or more columns, with pointers to the rows. It turns a full scan **O(n)** into a lookup O(log n).

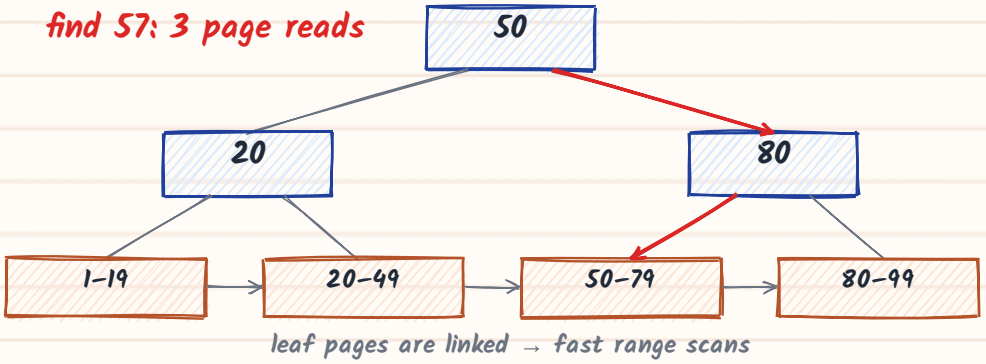
1 What an index is



→ Without index: full table scan

→ With index: jump to the key, follow the pointer

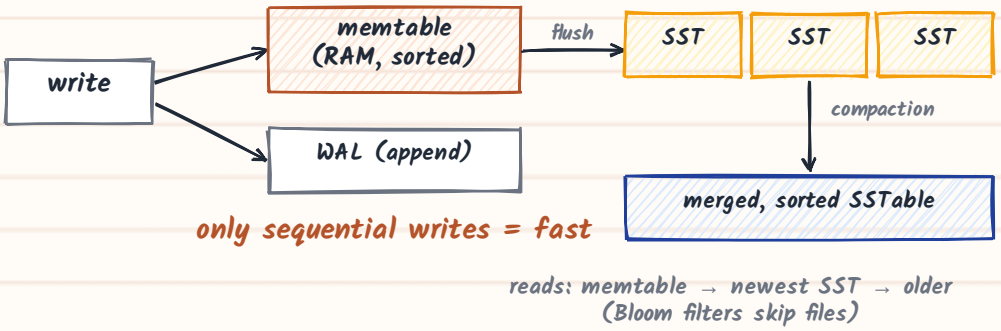
2 B-tree



- ✓ Wide, shallow tree of fixed-size pages (4–16 KB)
- ✓ Billions of rows → only 3–4 levels
- ✓ Leaves linked → fast range scans
- ✓ Updates pages **in place**

> **Cost**
> Random I/O on writes + page splits. Used by Postgres, MySQL, SQL Server.

4 LSM-tree (log-structured merge)



→ Write → **memtable** (RAM) + **WAL** (append)
→ Full memtable → **flush** to immutable SSTable
→ **Compaction** merges files, drops old versions
→ Read: memtable → newest SST → older; **Bloom filters** skip files

6 B-tree vs LSM-tree

	B-tree	LSM-tree
Writes	in place, random I/O	append, sequential (fast)
Reads	fast, predictable	may check several files
Range scans	great	good
Amplification	page rewrites	compaction rewrites
Used by	Postgres, MySQL	Cassandra, RocksDB, HBase
Best for	read-heavy, OLTP	write-heavy: feeds, logs, metrics

7 Using indexes well

```
CREATE INDEX idx_photos_user_time
  ON photos (user_id, created_at DESC);
```

→ **Composite**: leftmost-prefix rule

→ **Covering**: includes all needed columns → no table lookup

→ Every index costs **writes + space**: index real queries only

SnapLink

Profile page → **(user_id, created_at DESC)**. Feed timelines in Cassandra (LSM) because writes dominate.

8 Interview Corner

Q. Why is an indexed lookup fast?

A. The index is sorted: O(log n), a few page reads, instead of scanning every row.

Q. B-tree or LSM-tree?

A. B-tree for read-heavy, predictable reads. LSM for write-heavy workloads.

Q. How does an LSM tree keep reads fast?

A. Bloom filters skip files; compaction keeps the file count low.

Q. What is a composite index?

A. An index on several columns, usable by any leftmost prefix of them.

Q. What does an index cost?

A. Slower writes (every index is updated) and extra storage.

⚡ Revise in 60 seconds

- → Index = **sorted** + pointers
- → **O(n) → O(log n)**
- → **B-tree**: in place, reads
- → Leaves linked: ranges
- → **LSM**: memtable → SST
- → Compaction + Bloom
- → Composite: leftmost prefix
- → Index = write cost

Next lesson → SD-12
Replication: leader-follower, multi-leader, leaderless and quorums.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**